Counting at Scale

shard it, batch it, or estimate it

Watch the 4-min video

In one line: a single counter row becomes a **hot row**, so high-volume counts use **sharded counters** or **in-memory batching** (write-behind); unique counts use **HyperLogLog**, a 12 KB sketch with ~0.8% error that merges across hours and pages. Decide per metric whether it must be exact (money) or can be approximate (views).

1 A simple number on every page

A news site shows "1.2M reads" and "34,000 unique visitors today" on every article. A popular article gets 50,000 views a minute. `UPDATE articles SET views = views + 1` makes every request queue on one row lock; and counting unique visitors exactly means remembering every visitor ID for every page, every day. Two different problems: **throughput** for plain counts, **memory** for distinct counts.

2 Plain counts without a hot row

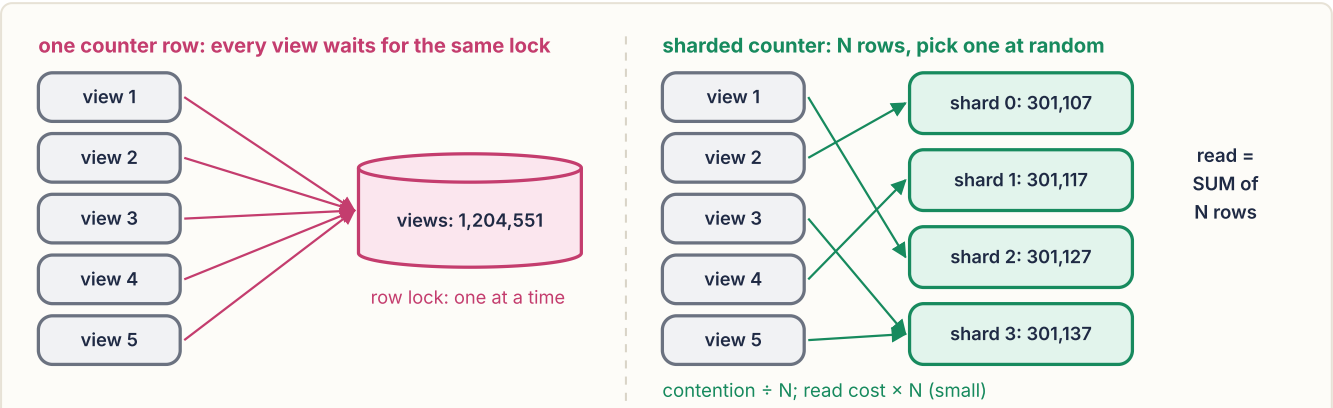


Figure 1. Spreading one logical counter across N rows divides lock contention by N; a read sums N small rows.

Every database has a per-row (or per-document) write ceiling: Firestore documents its limit as roughly one sustained write per second per document, which is why Google's docs teach **sharded counters**; a relational row serialises increments on its lock. Shard the counter into N rows (`counter_id, shard_no, n`), increment a random shard, and read with `SUM(n)`. Choose N from the peak write rate ÷ safe rate per row, and cache the sum for a few seconds.

An in-memory store helps too: Redis `INCR` is atomic and a single node handles on the order of 100,000 simple operations a second, but one key still lives on one node, so a truly viral counter may need sharded keys as well.

3 Write-behind: count in memory, flush in batches

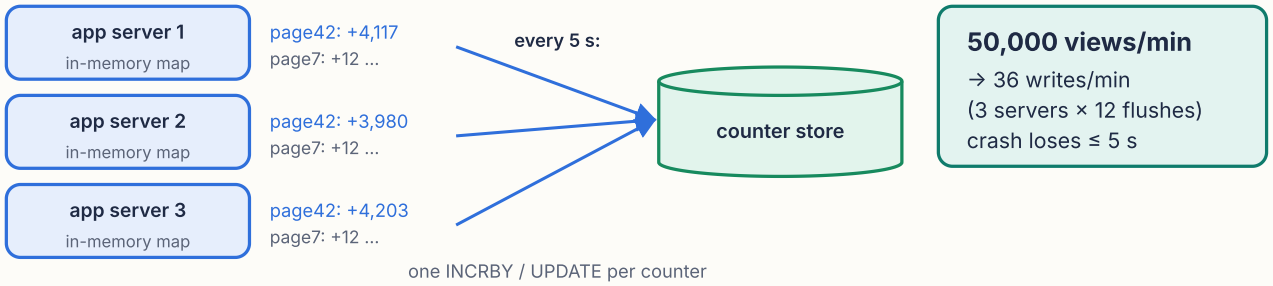


Figure 2. Each app server aggregates locally and sends one increment per counter per interval.

Each app server keeps a map of pending increments and flushes it every few seconds with one `INCRBY` or `UPDATE … SET n = n + 4117` per counter. 50,000 writes a minute become a few dozen. The cost: a crash loses up to one interval of counts, and displayed numbers lag by the interval. That is fine for views and likes counts, never for money. If counts must not be lost, publish view events to Kafka instead and let a stream job (lesson 32) aggregate per minute with checkpoints: the same batching, made durable.

4 Unique counts: HyperLogLog

Exact unique counts need the set of IDs. One million pages × 10,000 unique visitors × 16-byte IDs is about **160 GB per day**, and weekly uniques can't be computed by adding daily numbers (the same person visits on several days). **HyperLogLog** (Flajolet et al., 2007) estimates the number of distinct items in fixed memory.

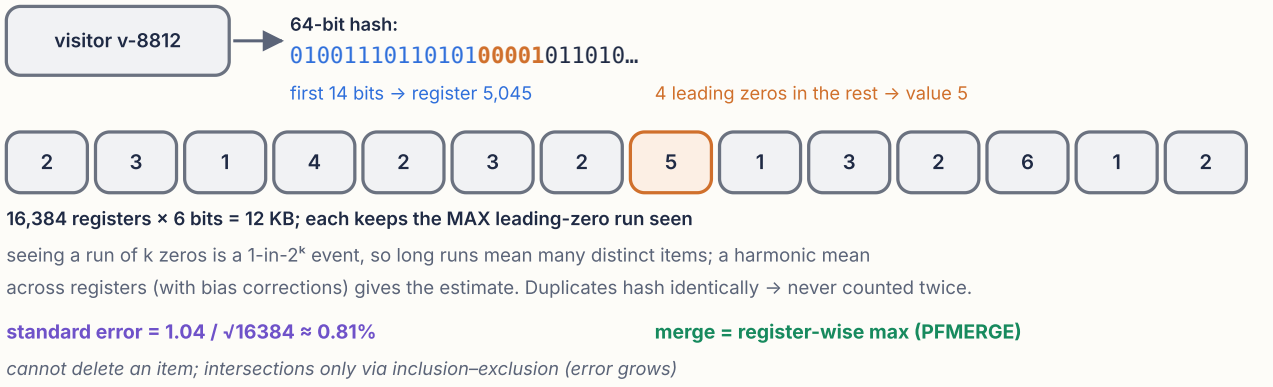


Figure 3. HyperLogLog as Redis implements it (p = 14). Rare long zero-runs in hashes reveal how many different items have been seen.

In Redis: `PFADD key item`, `PFCOUNT key` (also accepts several keys and returns the size of their union), and `PFMERGE dest k1 k2 …`. Small HLLs use a sparse encoding far below 12 KB; the dense form is capped at 12 KB however many items you add. Because merging takes the per-register maximum, hourly sketches combine into daily ones, daily into weekly, and page sketches into a site-wide one, with no double counting.

For **how often each item** appears (rather than how many distinct items), use a **Count-Min Sketch**: a small grid of counters with one hash per row; an item's estimate is the minimum of its counters, so it can overestimate (collisions) but never underestimate (lessons 39 and 40).

Metric	Exact?	Technique
Account balance, inventory, invoices	exact, durable	transactions, idempotency keys, ledgers
Likes (one per user)	exact set, approximate display	a row per (user, post) with a unique key; cached count
Page views, video plays	approximate, eventually consistent	sharded counters, write-behind, stream aggregation
Unique visitors	approximate	HyperLogLog per page per day, merged
Trending counts per item	approximate	Count-Min Sketch + heap (lesson 40)

WORKED EXAMPLE: UNIQUE VISITORS PER PAGE

Each page view runs `PFADD uv:page42:2026-10-31 v-8812`. Today's number is `PFCOUNT uv:page42:2026-10-31`; this week's is `PFCOUNT` over the seven daily keys (or `PFMERGE uv:page42:2026-W44` them once and cache). Each key is at most ~12 KB, so a million pages cost at most ~12 GB a day (less, thanks to sparse encoding for quiet pages), instead of storing billions of IDs, and every number is within about 1% (standard error 0.81%). Daily keys get a 35-day `EXPIRE`. The view count on the same page uses write-behind: each server flushes `INCRBY views:page42 n` every 5 seconds.

TRADE-OFFS

- More counter shards: more write throughput, slightly slower and less consistent reads.
- Longer flush intervals: fewer writes, but staler numbers and more loss on a crash.
- HyperLogLog: constant memory and mergeable, but ~1% error, no deletes, no list of who visited.
- Exact counting is always possible; the question is whether the metric is worth its cost.

COMMON MISTAKES

- One counter row for a viral item.
- Adding daily unique counts to get weekly uniques (counts people several times).
- Using write-behind or approximate counts for money or stock levels.
- Counting likes by incrementing a number, so double-taps and retries count twice.

5 Key terms

Hot row / key: one record receiving most of the writes

Write-behind: aggregate in memory, flush periodically

PFADD / PFCOUNT / PFMERGE: Redis HLL add, estimate, merge

Sharded counter: one count split over N rows, summed on read

HyperLogLog: fixed-size sketch estimating distinct counts

Count-Min Sketch: frequency estimates that only overestimate

6 Interview questions

> **Q1. How do you count unique users cheaply?** BASIC
> HyperLogLog: a fixed ~12 KB sketch that estimates distinct counts within about 1% and merges across hours or pages.

Q2. How do you count high-volume views without a hot row? BASIC

Shard the counter across rows and sum on read, or count in memory and flush periodically (write-behind).

Q3. Daily uniques are stored as numbers. Product wants weekly uniques. What now? SCENARIO

You can't add them; store a HyperLogLog per day instead, then PFCOUNT or PFMERGE the seven sketches.

Q4. How many counter shards do you need for 2,000 increments/s if one row handles ~200/s safely? SCENARIO

At least 10, more for headroom (say 20); reads sum 20 rows and are cached briefly.

Q5. Why can HyperLogLog merge without double-counting? TRICKY

Each register stores the maximum zero-run for its hash bucket; the same visitor hashes identically everywhere, so max-merging equals adding all items to one sketch.

Q6. When is write-behind unacceptable? TRICKY

When losing a few seconds of increments matters (payments, stock, quotas); use transactional updates or durable event logs instead.

Next: SD-Q-014 · Probabilistic Data Structures backendroadmap.com